

Class 07

Aarav Prasad

Table of contents

Background	1
k-means clustering	1
Hierarchical Clustering	7
Principal Component Analysis (PCA)	9
Analysis of UK food data	9
Data Import	9
Tidy data	10
Exploratory analysis	11
PCA to the rescue	16

Background

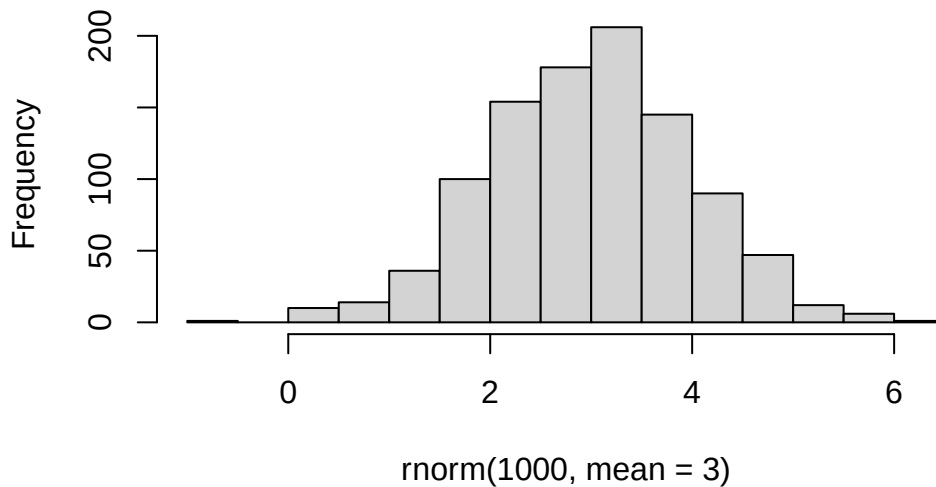
We will be looking at different ways to cluster data.

k-means clustering

We will use `kmeans()` for k-means clustering, the base r function for this. We will first use some easy sample data, using `rnorm()`.

```
hist(rnorm(1000, mean=3))
```

Histogram of rnorm(1000, mean = 3)



```
x <- c(rnorm(30, mean = -3), rnorm(30, mean = 3))
x
```

```
[1] -3.775515 -2.162455 -2.297600 -3.623153 -4.366985 -2.630220 -2.581334
[8] -2.264769 -2.148424 -2.380392 -4.143861 -3.815141 -2.770462 -1.286723
[15] -1.377434 -3.355480 -2.297469 -4.403207 -3.365535 -2.722024 -2.158753
[22] -2.141311 -4.781859 -4.784341 -1.620992 -3.681468 -5.094706 -1.277517
[29] -3.797682 -2.847092  3.360475  3.190490  1.266964  2.799126  2.127634
[36]  4.884297  3.223005  2.139973  2.884135  2.906993  3.102871  4.277519
[43]  2.849783  4.446749  4.537901  3.572004  2.629058  2.654319  3.242235
[50]  3.655185  2.688383  2.504454  4.236883  3.115916  3.910912  2.206101
[57]  3.180521  2.269481  2.745600  1.853599
```

```
z <- cbind(x = x, y = rev(x))
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z, centers = 2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

```
      x      y
1  3.082086 -2.998463
2 -2.998463  3.082086
```

Clustering vector:

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Within cluster sum of squares by cluster:

```
[1] 55.21044 55.21044
(between_SS / total_SS = 90.9 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"       "withinss"    "tot.withinss"
[6] "betweenss"    "size"        "iter"       "ifault"      "
```

```
attributes(km)
```

\$names

```
[1] "cluster"      "centers"      "totss"       "withinss"    "tot.withinss"
[6] "betweenss"    "size"        "iter"       "ifault"      "
```

\$class

```
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What component of your result object details cluster assignment/membership?

```
km$cluster
```

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

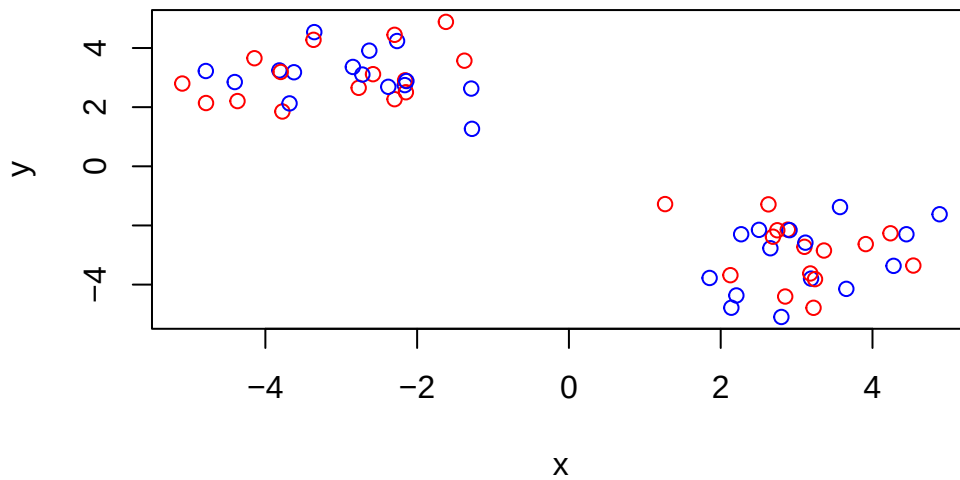
Q. What component of your result object details cluster center?

```
km$centers
```

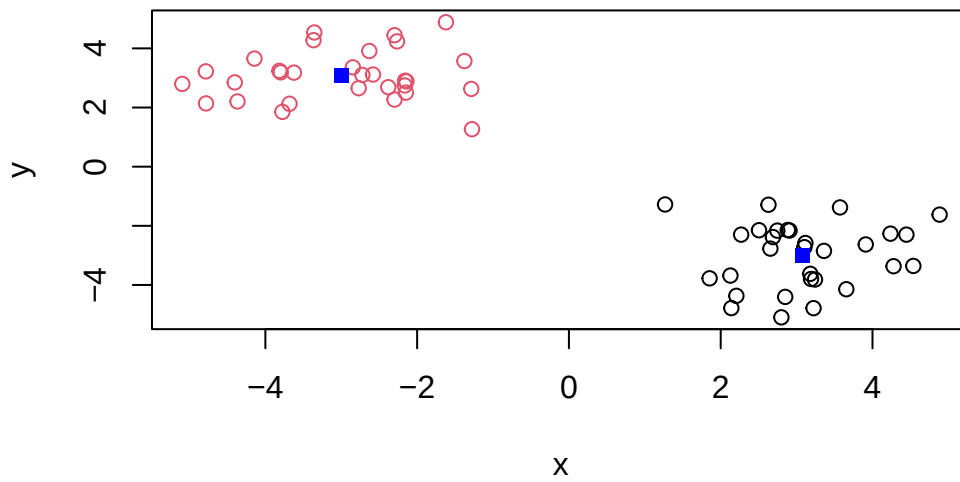
	x	y
1	3.082086	-2.998463
2	-2.998463	3.082086

Q. Plot `z` colored by the kmeans cluster assignment and add cluster centers as blue points

```
plot(z, col = c("red", "blue"))
```

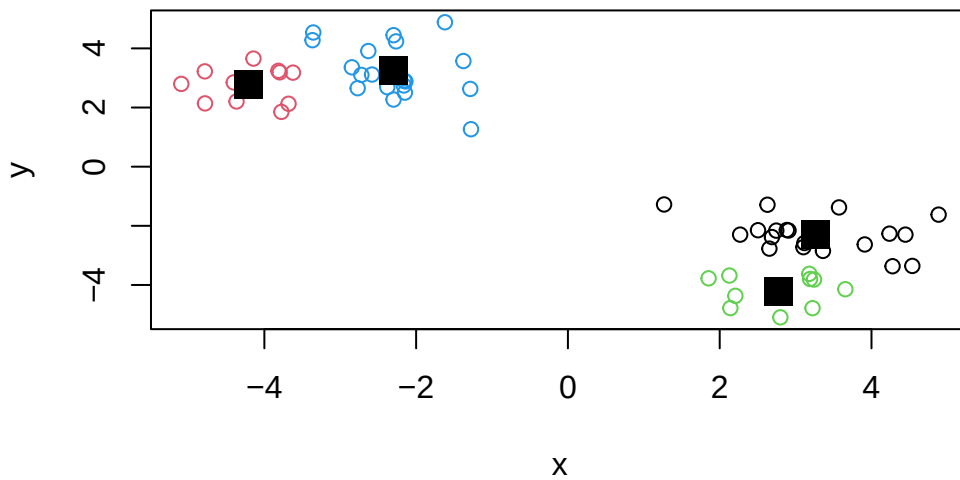


```
plot(z, col = km$cluster)
points(km$centers, col = "blue", pch = 15)
```



Q. Run a k-means clustering and plot the results asking for 4 clusters?

```
km4 <- kmeans(z, centers = 4)
plot(z, col = km4$cluster)
points(km4$centers, col = "black" , pch =15, cex = 2)
```

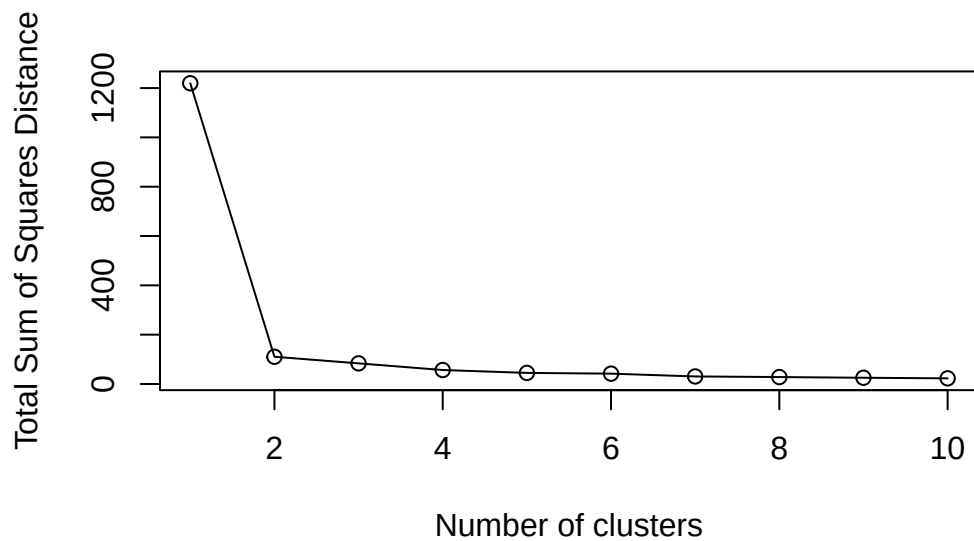


Note You need to tell k-means the number of clusters (i.e. set `centers=2`)!!

One approach is to try different values for `centers` and then pick the best...

```
ans <- NULL
for(i in 1:10){
  km <- kmeans(z, centers=i)
  ans <- c(ans, km$tot.withinss)
}

plot(ans, typ = "o", xlab = "Number of clusters", ylab= "Total Sum of Squares Distance")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

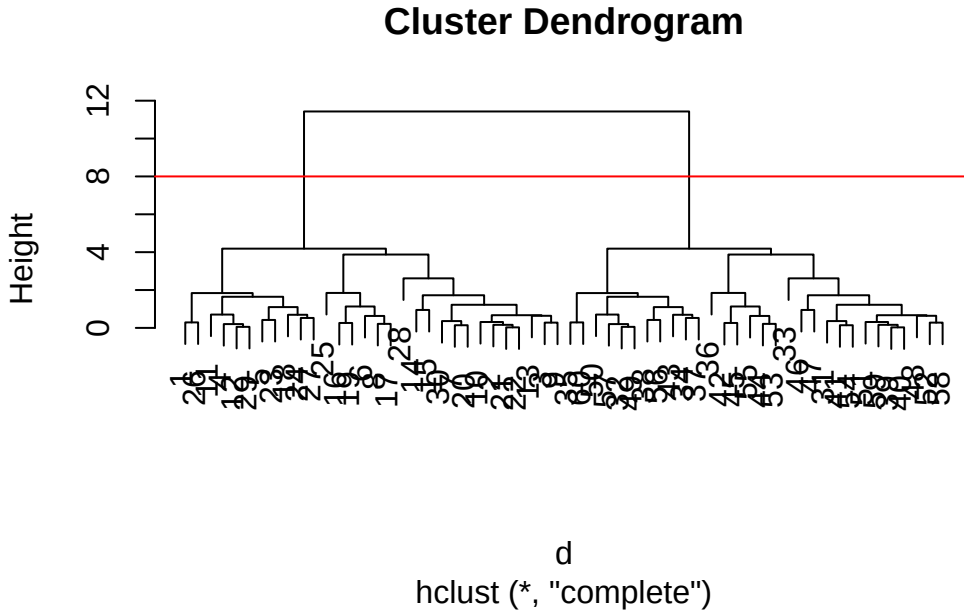
Call:

```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8, col = "red")
```



Once we have our `hclust` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

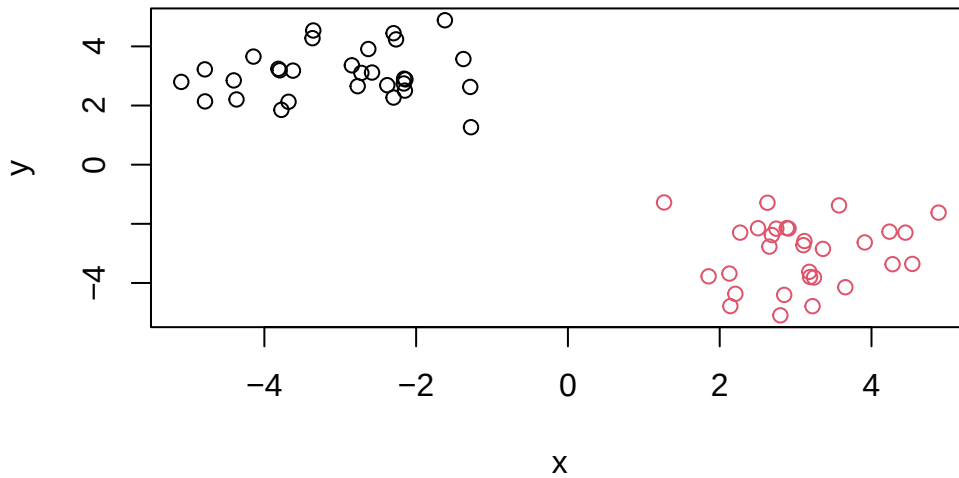
[illegible]

```
cutree(hc, k=4)
```

[1] 1 2 2 1 1 2 2 2 2 2 1 1 2 2 2 2 2 1 2 2 2 2 1 1 2 1 1 2 1 2 3 4 3 4 4 3 4 4
[39] 3 3 3 3 4 3 3 3 3 3 4 4 3 3 3 3 3 4 4 3 3 4

Q. Make a plot of $\mathbf{z}$ with your hclust results (i.e. colored by cluster membership)

```
grps <- cutree(hc, k = 2)
plot(z, col = grps)
```

Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()` function for import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
```

Q1: How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

There are 17 rows and 5 columns. `dim(x)`, `ncol(x)`, and `nrow(x)` can be used

```
dim(x)
```

```
[1] 17  5
```

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

Tidy data

```
rownames(x) <- x[,1]  
x <- x[,-1]  
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17  4
```

```
x <- read.csv(url, row.names=1)  
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

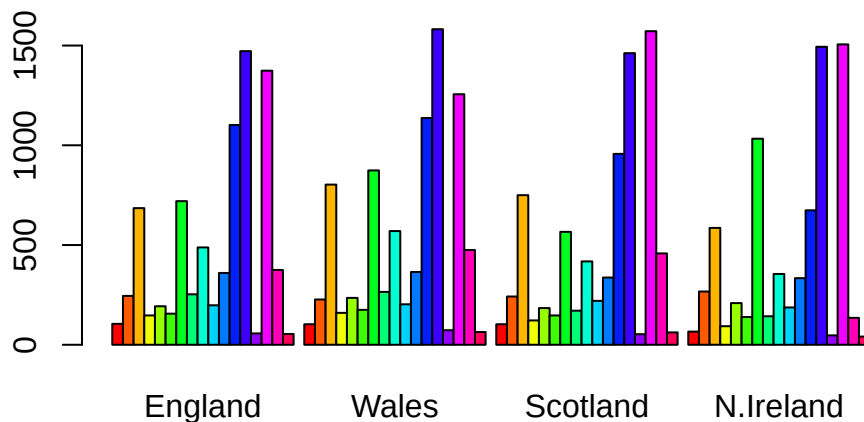
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer `row.names = 1` because it’s cleaner and fixes the issue immediately. The manual method is better if the data is already loaded or needs inspection, but it is destructive and keeps removing columns.

Exploratory analysis

Spotting major differences or threads

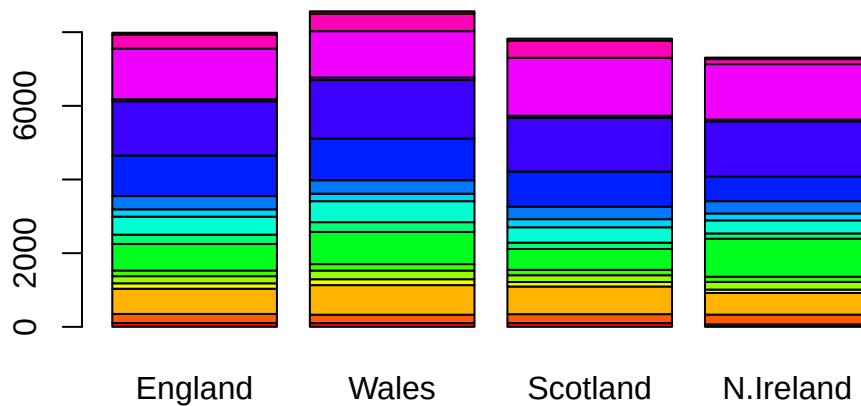
```
# Using base R
barplot(as.matrix(x), beside = T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

Setting `beside` to `false` (or leaving it out),

```
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

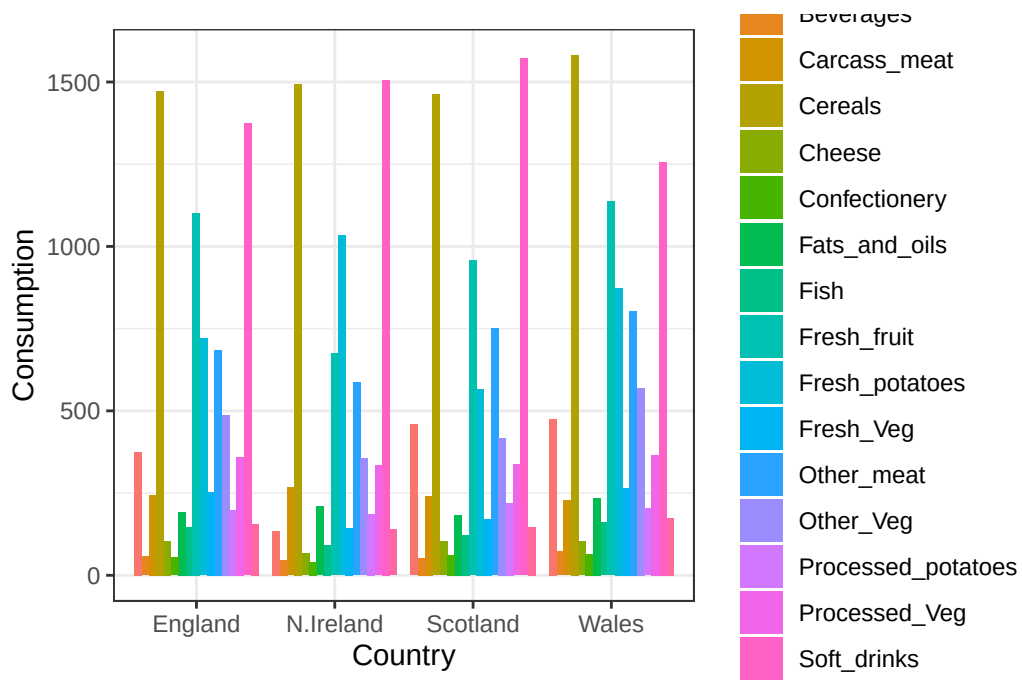
dim(x_long)
```

```
[1] 68 3
```

```
head(x_long)
```

```
# A tibble: 6 x 3
  Food      Country Consumption
  <chr>    <chr>      <int>
1 "Cheese"  England      105
2 "Cheese"  Wales        103
3 "Cheese"  Scotland     103
4 "Cheese"  N.Ireland     66
5 "Carcass_meat " England      245
6 "Carcass_meat " Wales        227
```

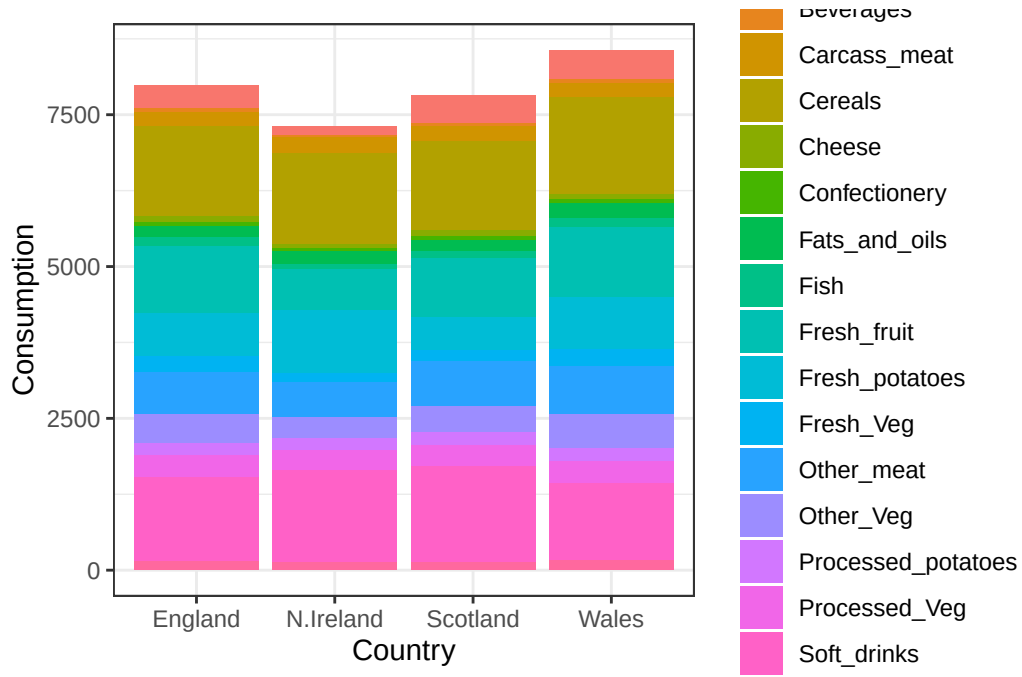
```
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

Removing the `geom_col` argument `position="dodge"` creates the stacked barplot figure.

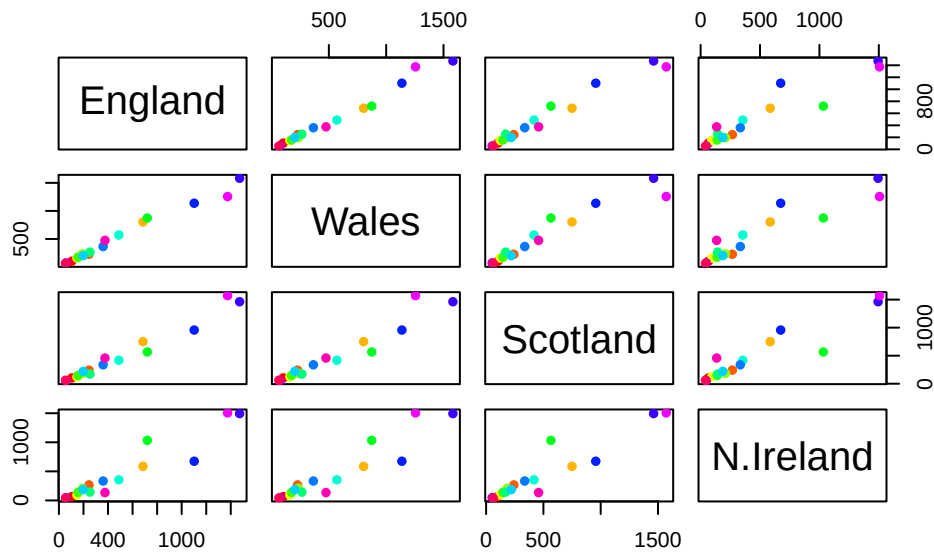
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col() +
  theme_bw()
```



Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

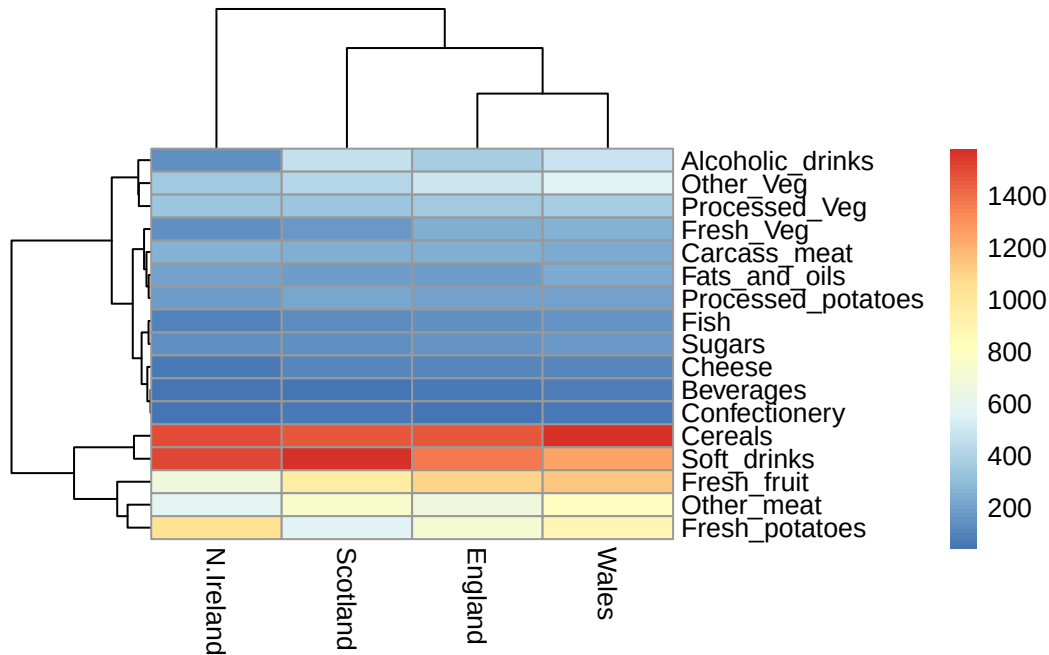
If a point lies on the diagonal for a plot that means the countries have equal values for that food.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Key-point: Even relatively small datasets can prove challenging to interpret.

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object

```
# Use the prcomp() PCA function
pca <- prcomp( t(x) )
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00


```
Cumulative Proportion    0.6744    0.9650    1.00000 1.000e+00
```

The returned `pca` object has components that we can use to make our main result figures:

```
attributes(pca)
```

```
$names
```

```
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

```
$class
```

```
[1] "prcomp"
```

The main result figure from this analysis is called a “PC score plot” or “ordination plot” “PC plot” or “PC1 vs PC2 plot”

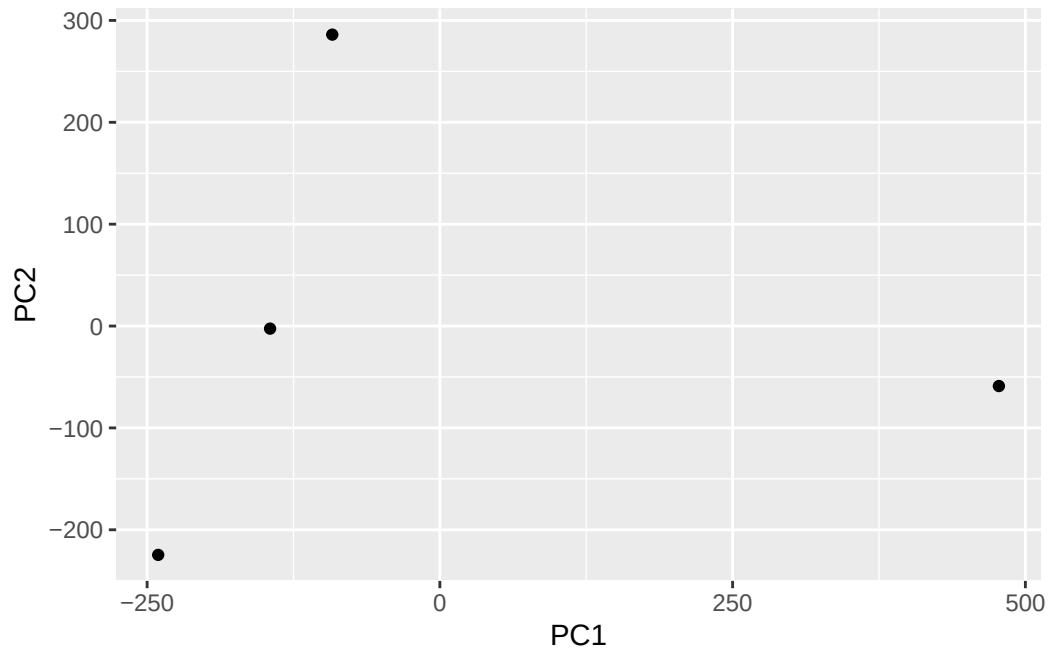
This plot shows how samples (in this case countries) relate to each other along our new PC axis.

This is our new “reduced-dimensional space”. In this case 2 dimensions, PC1 and PC2, that capture most of the variance in the original 17 dimensional data-set.

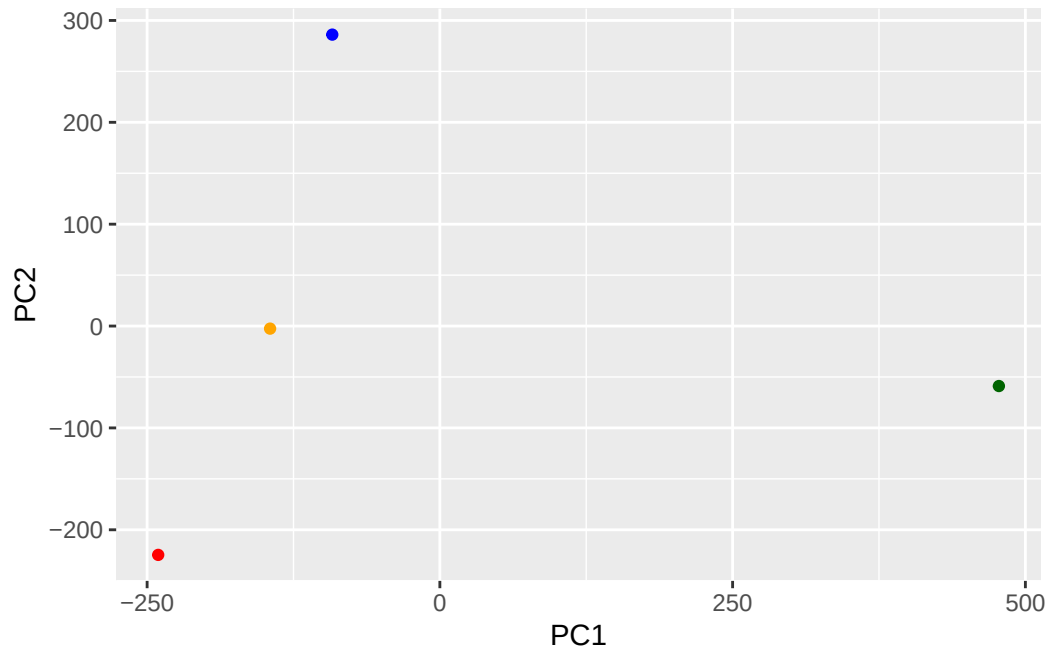
```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	-9.152022e-15
Wales	-240.52915	-224.646925	-56.475555	5.560040e-13
Scotland	-91.86934	286.081786	-44.415495	-6.638419e-13
N.Ireland	477.39164	-58.901862	-4.877895	1.329771e-13

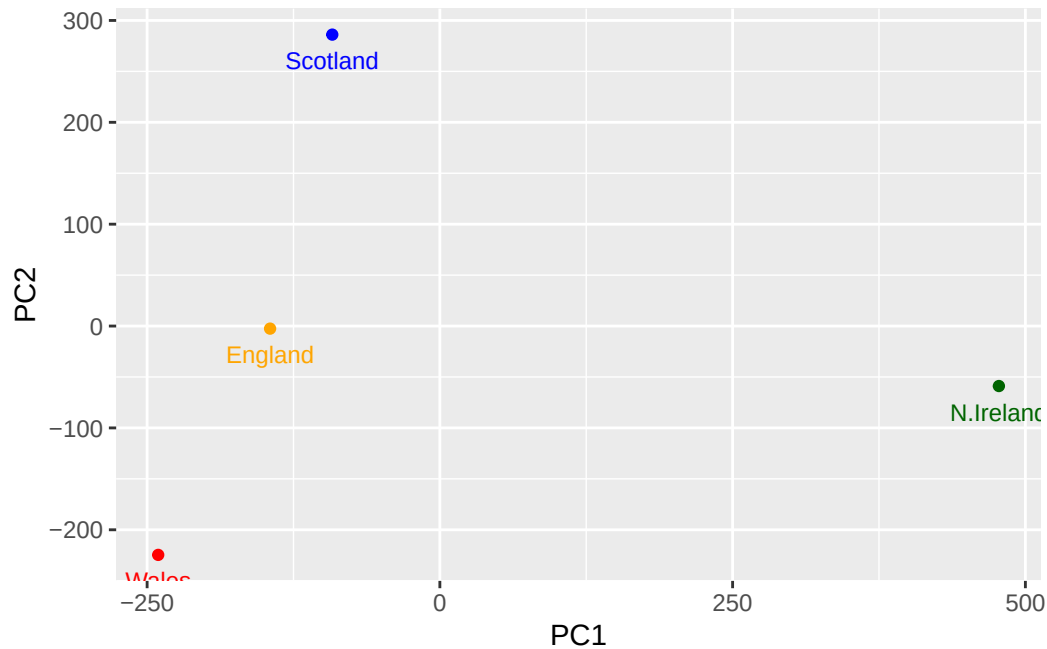
```
ggplot(pca$x) +  
  aes(PC1, PC2) +  
  geom_point()
```



```
mycols <- c("orange", "red", "blue", "darkgreen")
ggplot(pca$x) +
  aes(PC1, PC2) +
  geom_point(col = mycols)
```



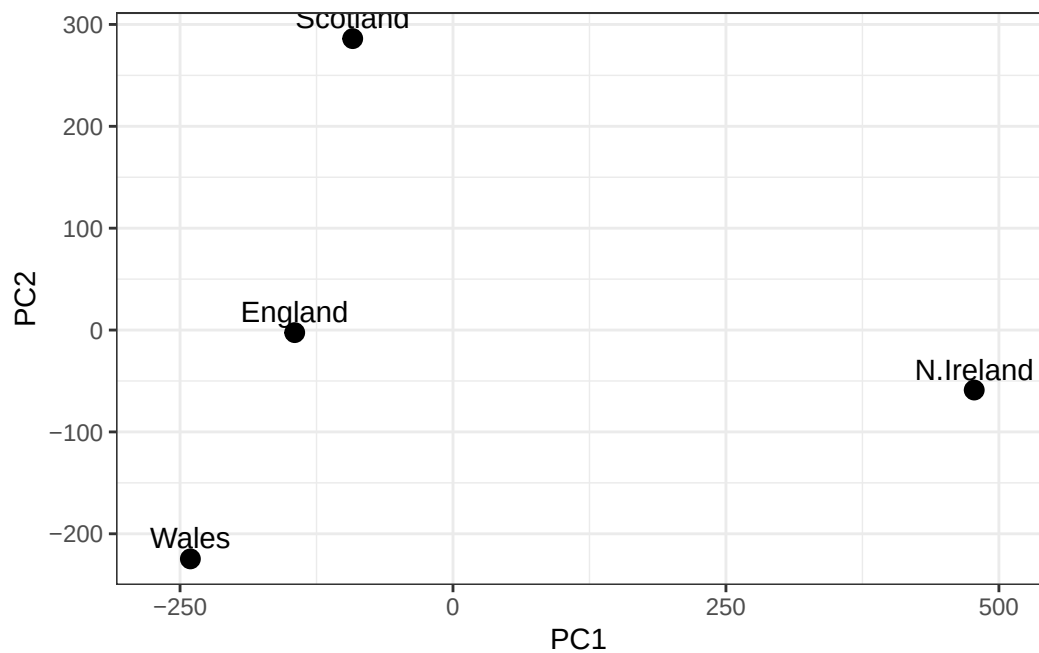
```
ggplot(pca$x) +  
  aes(PC1, PC2, label=row.names(pca$x)) +  
  geom_point(col = mycols) +  
  geom_text(size=3, vjust=2, col=mycols)
```



Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

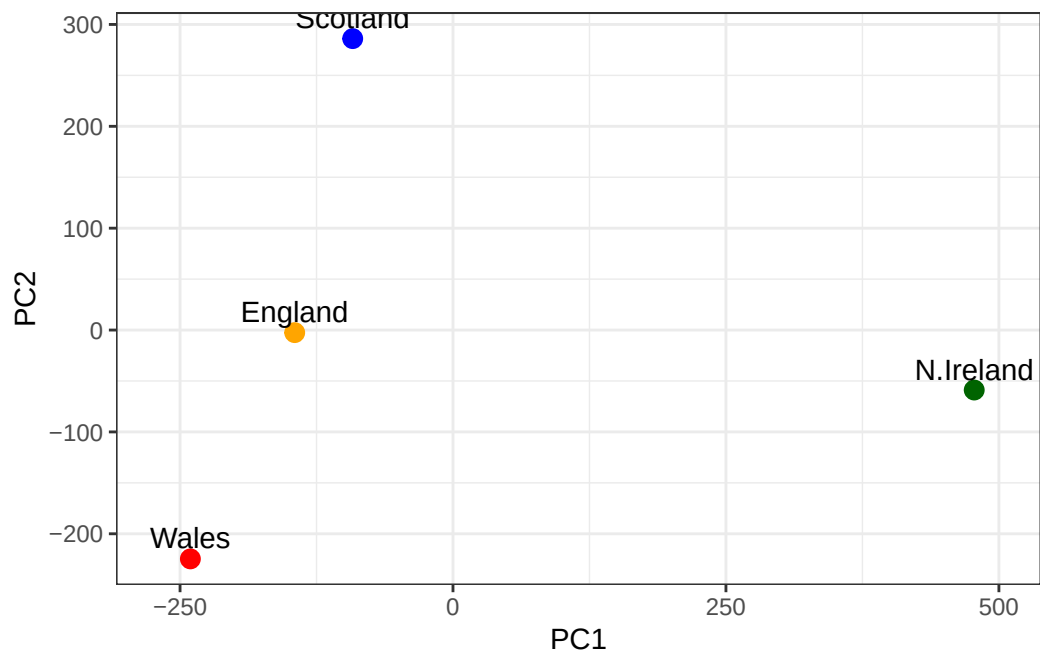
# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



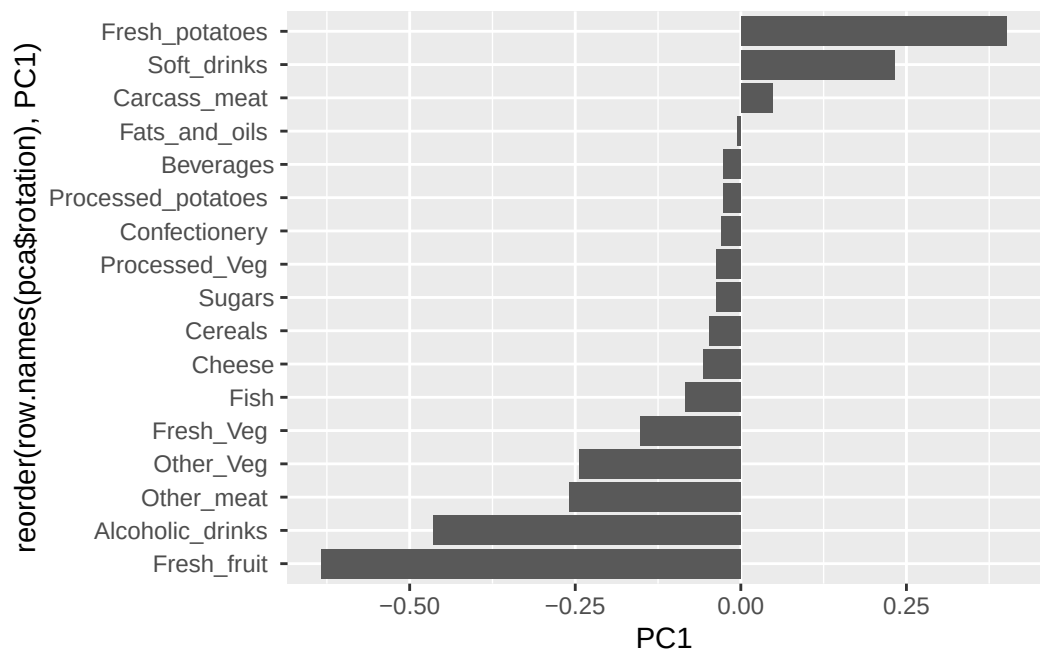
Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3, col = mycols) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



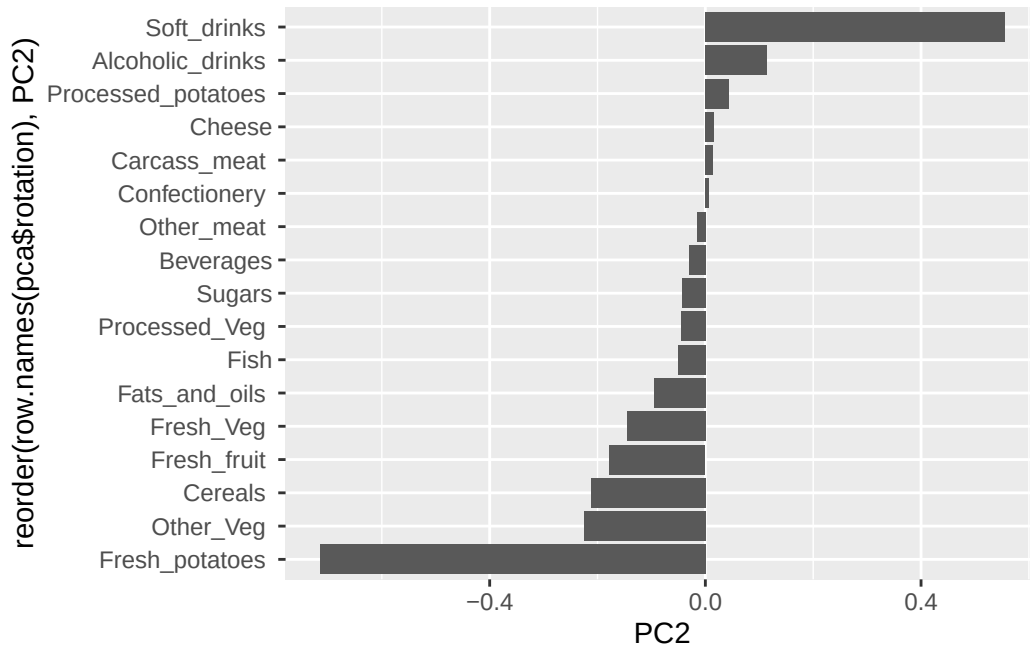
```
ggplot(pca$rotation) +
  aes(PC1, reorder(row.names(pca$rotation), PC1)) +
  geom_col()
```



Q9. Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

Fresh potatoes and soft drinks feature prominently, PC2 tells us about the next biggest pattern of variation independent from PC1.

```
ggplot(pca$rotation) +  
  aes(PC2, reorder(row.names(pca$rotation), PC2)) +  
  geom_col()
```



A **PC score plot** shows how samples are positioned along principal components (revealing clustering and variation), while a **loading plot** shows which variables drive those components; together, the loadings explain *why* samples appear where they do in the score plot.